5. RAG 챗봇 실습

RAG 챗봇은 외부 데이터베이스에서 관련 정보를 검색하고 이를 기반으로 답변을 생성하여 사용자에게 보다 정확하고 신뢰성 있는 응답을 제공한다. RAG 프로세스는 인덱싱 과정과 쿼리 과정으로 나눌 수 있다.

1 검색 증강 생성 개요

1.1 텍스트 임베딩
1.2 코사인 유사도
1.3 랭체인 임베딩 API 활용

2 문서 로더

2.1 웹 페이지 로더
2.2 PDF 로더
2.3 CSV 로더

3 텍스트 분할

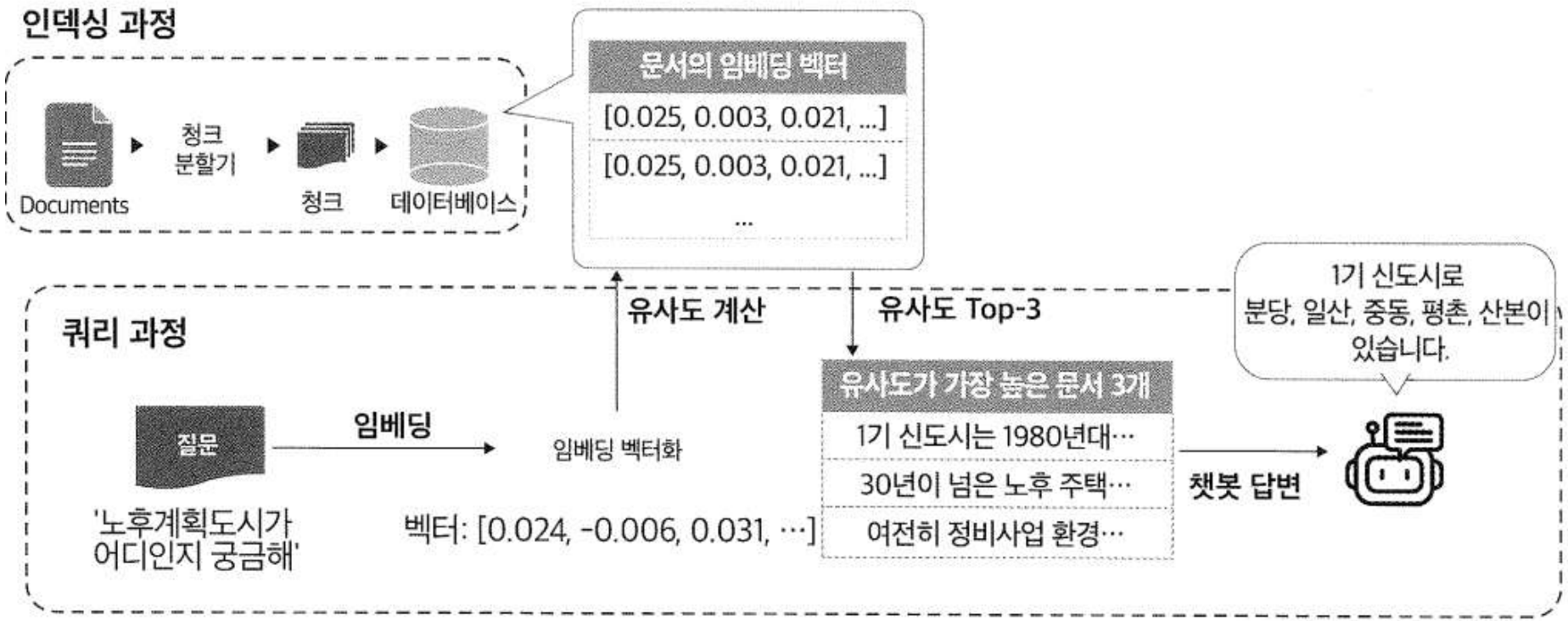
3.1 길이와 구분자로 분할하는 재귀적 문자 텍스트 분할
3.2 의미 기반으로 분할하는 시맨틱 청킹

4 벡터 데이터베이스

4.1 크로마
4.2 파이스

5 RAG 챗봇 실습

5.1 RAG 챗봇 구현
5.2 챗봇에 스트림릿 UI 적용



인덱싱 과정

인덱싱 과정(Index Process)에서는 문서를 수집하고, 검색이 용이하도록 데이터베이스에 적재

1. 문서 준비 및 분할:

문서를 수집하고 이를 작은 조각(Chunks)으로 나눔. 이 청크들은 의미 단위로 적절한 크기로 설정하는 것이 중요함. 너무 작으면 정보가 부족하고 너무 크면 검색 정확도가 떨어짐.

2. 임베딩 생성 및 저장:

청크를 임베딩 모델을 사용해 벡터로 변환. 임베딩은 청크를 의미를 숫자로 표현한 벡터로, 벡터 데이터베이스에 저장되어 나중에 관련 정보를 빠르게 찾는 데 사용됨.

3. 데이터베이스 적재 및 관리:

변환된 임베딩 벡터는 벡터 데이터베이스에 저장되며, 이를 통해 효율적인 검색이 가능함.

쿼리 과정

쿼리 과정(Query Process)은 사용자가 질문을 입력하면 관련 정보를 실시간으로 검색하고 답변을 생성

1. **질문 입력 및 변환(임베딩):** 사용자가 질문을 입력하면, 이 질문을 (DB 검색을 위해) 벡터로 변환. 이 과정에서 질문은 이전 대화 맥락을 반영해 재 작성될 수 있다.
2. **검색 및 재정렬**: 변환된 질문 벡터는 DB에서 관련성 높은 문서 청크들을 검색하는데 사용. 유사도 점수에 따라 상위 개의 K개의 청크를 선택. 필요시 다시 검색.
3. **프롬프트 템플릿 설정**: 사용자 질문에 적절한 답변을 생성하도록 프롬프트 템플릿을 설정. 프롬프트 템플릿은 입력된 질문과 검색된 문서 청크를 결합해 대화형 응답을 만든다.
4. **문맥 구성(챗봇 체인 구성, 메모리 설정)**: 검색된 청크들을 조합해 질문에 맞는 문맥을 형성. 언어 모델의 토큰 수 제한을 고려하여 필요한 부분만 포함.
5. **답변 생성 및 응답 제공(챗봇 실행)**: 구성된 문맥과 질문을 함께 대규모 언어 모델에 입력해 답변을 생성. 생성된 답변은 필요한 형식과 스타일로 정제하여 사용자에게 최종 응답으로 제공.

5.1 RAG 챗봇 구현

'2024 KB 부동산 보고서'를 기반으로 답변하는 RAG 챗봇을 구현하자.

① 이 문서를 Chunk들로 분할하고, 각 청크를 임베딩 벡터로 변환하여 저장하는 인덱싱 과정을 수행. 이 단계는 DB 준비 작업이며 챗봇 실행 전에 완료해야 한다.

② 챗봇이 실행되고 사용자가 질의를 입력하면, 질의를 임베딩하여 벡터로 변환한 후 미리 저장된 '2024 KB 부동산보고서'의 임베딩 벡터들과 유사도를 계산.(쿼리 과정, 유사도가 높을수록 사용자 질문과 관련된 문서일 가능성이 크다는 의미) 유사도 점수를 바탕으로 가장 관련성이 높은 상위 3개의 문서를 선택하여 사용자의 질문과 함께 오픈AI API의 프롬프트로 전달하여 최종 답변을 생성.

ch02_RAG_CHATBOT.ipynb 참고

```python
# 라이브러리 불러오기
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain.schema.output_parser import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableWithMessageHistory
from langchain.memory import ChatMessageHistory

# PDF 문서 로드 및 텍스트 분할
loader = PyPDFLoader("/content/drive/MyDrive/langchain-tutorial/Ch02. RAG/Data/2024_KB_부동산_보고서_최종.pdf")
documents = loader.load()  # 문서 로드
```

"2024_KB_부동산_보고서_최종.pdf"로 대체 가능

```python
# 텍스트 분할 설정: 청크 크기와 겹침 설정
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = text_splitter.split_documents(documents)

# 분할된 청크 수
print('분할된 청크의 수:', len(chunks))
```

```
분할된 청크의 수: 135
```

```python
# 임베딩 생성 및 Chroma 데이터베이스 저장
embedding_function = OpenAIEmbeddings()  # 임베딩 모델 설정

persist_directory = "/content/drive/MyDrive/langchain-tutorial/Ch02. RAG/directory/chroma"

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embedding_function,
    persist_directory=persist_directory  # 데이터베이스 저장 경로
)


print('문서의 수:', vectorstore._collection.count())
```

"./chroma_db"로 경로 대체 가능

쿼리 과정

```python
# 검색 및 재정렬
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})  # 관련 문서 상위 3개 검색 설정

# 프롬프트 템플릿 설정: 사용자 질문에 대한 답변을 생성하기 위한 템플릿
template = """당신은 KB 부동산 보고서 전문가입니다. 다음 정보를 바탕으로 사용자의 질문에 답변해주세요.

컨텍스트: {context}
"""
prompt = ChatPromptTemplate.from_messages(
    [
        ("system", template),
        ("placeholder", "{chat_history}"),
        ("human", "{question}")
    ]
)  # 템플릿 초기화
model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)  # AI 모델 설정
```

```python
print(prompt.format(context="컨텍스트 예시", chat_history=["대화 기록 예시1", "대화 기록 예시2"], question="질문 예시"))
```

```
System: 당신은 KB 부동산 보고서 전문가입니다. 다음 정보를 바탕으로 사용자의 질문에 답변해주세요.

컨텍스트: 컨텍스트 예시

Human: 대화 기록 예시1
Human: 대화 기록 예시2
Human: 질문 예시
```

```python
# 문서 형식 변환 함수 정의
def format_docs(docs):
    return "₩n₩n".join(doc.page_content for doc in docs)  # 문서 내용을 줄바꿈으로 연결


# 체인 구성: 검색한 문서를 프롬프트에 연결하고 모델을 통해 응답 생성
chain = (
    RunnablePassthrough.assign(
        context=lambda x: format_docs(retriever.invoke(x["question"]))
    )  # 검색된 문서를 연결하여 전달
    | prompt
    | model
    | StrOutputParser()  # 결과를 문자열로 변환
)
```

```python
# 대화 기록을 유지하기 위한 메모리 설정
chat_history = ChatMessageHistory()
chain_with_memory = RunnableWithMessageHistory(
    chain,
    lambda session_id: chat_history,  # 세션 ID별 대화 기록 생성
    input_messages_key="question",
    history_messages_key="chat_history",
)
```

```python
# 챗봇 실행 함수 정의
def chat_with_bot():
    session_id = "user_session"
    print("KB 부동산 보고서 챗봇입니다. 질문해 주세요. (종료하려면 'quit' 입력)")
    while True:
        user_input = input("사용자: ")
        if user_input.lower() == 'quit':
            break

        response = chain_with_memory.invoke(
            {"question": user_input},
            {"configurable": {"session_id": session_id}}
        )

        print("챗봇:", response)

# 메인 실행
if __name__ == "__main__":
    chat_with_bot()
```

KB 부동산 보고서 챗봇입니다. 질문해 주세요. (종료하려면 'quit' 입력)
사용자: 랭체인 챗봇 만들기에 대해 실습중입니다.
챗봇: 랭체인 챗봇 만들기는 흥미로운 주제입니다! 챗봇을 만들 때 어떤 언어나 플랫폼을 사용하고 계신가요? 어떤 종류의 챗봇을 만들고 계신지 궁금합니다. 도움이 필요하시면 언제든지 물어보세요. 함께 응원하겠습니다!
사용자: 방금 제가 뭐라고 이야기했나요?

챗봇: 귀하가 방금 말씀하신 내용은 "랭체인 챗봇 만들기에 대해 실습중입니다." 였습니다. 다른 질문이 있으시면 도와드릴 수 있습니다.
사용자: 수도권 주택 매매 전망
챗봇: 2024년 KB 부동산 보고서에 따르면, 수도권 주택시장은 전반적인 침체 상황이 지속되고 있습니다. 과거와는 다르게 높은 기준금리와 주택 매매가격, DSR 규제 등으로 매수자들의 구매 여력이 회복되지 못하고 있습니다. 그러나 2023년 이후 정부의 다양한 규제 완화로 매도자들의 기대 심리는 높아지고 있습니다. ... (중간 생략) ...
사용자: 노후계획도시가 어디인지 궁금해
챗봇: 노후계획도시는 1기 신도시로 불리는 지역들을 가리키는 용어입니다. 1기 신도시는 1980년대 말에 '주택 200만 호 건설' 계획에 따라 성남시 분당, 고양시 일산, 부천시 중동, 안양시 평촌, 군포시 산본의 5개 지역에 대규모로 건설된 지역들을 말합니다. 이러한 지역들은 20년 이상 경과한 100만㎡ 이상 택지에 대한 정비사업이 필요한 지역으로 분류됩니다.
따라서, 노후계획도시로 불리는 지역은 성남시 분당, 고양시 일산, 부천시 중동, 안양시 평촌, 군포시 산본입니다. 이러한 지역들은 오랜 기간에 걸쳐 건설되어 왔으며, 재건축과 정비사업이 필요한 지역으로 주목받고 있습니다.
사용자: quit

5.2 챗봇에 스트림릿 UI 적용

완성한 챗봇에 직관적인 UI를 적용하면 사용자가 더욱 편리하게 정보를 검색하고 응답 받을 수 있다.

챗봇에 손쉽게 사용자 인터페이스(UI)를 적용할 수 있는 파이썬 라이브러리로 스트림릿(Streamlit)이 있다. 스트림릿을 활용하면 간단한 코드만으로 대화형 UI를 만들 수 있다.

앞서 만든 챗봇에 간단한스트림릿 UI를 적용해 보자.

스트림릿 싸이트 방문: https://streamlit.io/

```
# 라이브러리 설치
!pip install streamlit pyngrok
```

① Streamlit 설치

스트림릿을 사용하면 웹 대시보드를 쉽게 만들 수 있다. HTML이나 CSS같은 웹 개발 지식이 없어도 순수 파이썬 코드만으로 데이터 분석 결과를 웹에서 즉시 시각화 가능.

② Pyngrok 설치

Pyngrok은 로컬에서 실행 중인 서버를 외부 인터넷에서도 접근할 수 있도록 해주는 터널 링 도구. 구글코랩이나 로컬 환경에서 개발한 웹 애플리케이션을 다른 사람들과 공유할 때 유용하다.
Pyngrok은 내부적으로 'http://localhost:8501'과 같은 로컬 주소를 https://[무작위주소].ngrok-free.app'으로 변환하여 외부에서도 접근할 수 있도록 한다.

ngrok 인증 발급 및 인증키 설정

ngrok을 사용하려면, 먼저 다음과 같은 단계에 따라 인증키(Authtoken)를 발급받아야 한다.

① ngrok 회원 가입

ngrok 공식 웹사이트(https://ngrok.com/)에 접속하여 회원 가입.

② 대시보드에서 인증키 확인

로그인한 후 ngrok 대시보드로 이동. 왼쪽 메뉴에서 [Your Authtoken] 항목을 클릭하면 본인의 인증키 확인 가능.

[Your Authtoken] 화면에서 [Copy] 버튼을 클릭하여 인증키를 복사. 그리고 다음 코드를 코드셀에 입력하여 인증키 등록.

```
# ngrok 인증키 설정
!ngrok config add-authtoken <본인 인증키>
```

```
!ngrok config add-authtoken 2tnwXk5jQ5uWVBnwy30u8Mdmu8v_5eAtrJHxGZRiGGKqsqHsh
```

app.py 파일을 만든다.

```python
# app.py
import os
import streamlit as st
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain.schema.output_parser import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableWithMessageHistory
from langchain.memory import ChatMessageHistory

# 환경 변수 로드
load_dotenv("/content/.env")                    # load_dotenv()로 대체 가능
api_key = os.getenv("OPENAI_API_KEY")

# PDF 처리 함수
@st.cache_resource
def process_pdf():
    loader = PyPDFLoader("/content/drive/MyDrive/langchain-tutorial/Ch02. RAG/Data/2024_KB_부동산_보고서_최종.pdf")
    documents = loader.load()
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
    return text_splitter.split_documents(documents)

# 벡터 스토어 초기화
@st.cache_resource
def initialize_vectorstore():
    chunks = process_pdf()
    embeddings = OpenAIEmbeddings(openai_api_key=api_key)
    return Chroma.from_documents(chunks, embeddings)
```

코드 보완 필요:

저장된 vectorstore가 존재하는지 체크하고, 존재하면 새로 만들지 않고 있는 벡터 DB를 로드하여 return하는 코드 필요. 안 그러면, 매번 로드, 청킹, 임베딩하여 생성하는 낭비를 해야 한다. Streamlit은 reload가 잦다.

```python
# 체인 초기화
@st.cache_resource
def initialize_chain():
    vectorstore = initialize_vectorstore()
    retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

    template = """당신은 KB 부동산 보고서 전문가입니다. 다음 정보를 바탕으로 사용자의 질문에 답변해주세요.

    컨텍스트: {context}
    """
    prompt = ChatPromptTemplate.from_messages([
        ("system", template),
        ("placeholder", "{chat_history}"),
        ("human", "{question}")
    ])

    model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0, openai_api_key=api_key)

    def format_docs(docs):
        return "₩n₩n".join(doc.page_content for doc in docs)

    base_chain = (
        RunnablePassthrough.assign(
            context=lambda x: format_docs(retriever.invoke(x["question"]))
        )
        | prompt
        | model
        | StrOutputParser()
    )

    return RunnableWithMessageHistory(
        base_chain,
        lambda session_id: ChatMessageHistory(),
        input_messages_key="question",
        history_messages_key="chat_history",
    )
```

```python
# Streamlit UI
def main():
    st.set_page_config(page_title="KB 부동산 보고서 챗봇", page_icon="🏠")
    st.title("🏠 KB 부동산 보고서 AI 어드바이저")
    st.caption("2024 KB 부동산 보고서 기반 질의응답 시스템")

    # 세션 상태 초기화
    if "messages" not in st.session_state:
        st.session_state.messages = []

    # 채팅 기록 표시
    for message in st.session_state.messages:
        with st.chat_message(message["role"]):
            st.markdown(message["content"])

    # 사용자 입력 처리
    if prompt := st.chat_input("부동산 관련 질문을 입력하세요"):
        # 사용자 메시지 표시
        with st.chat_message("user"):
            st.markdown(prompt)
        st.session_state.messages.append({"role": "user", "content": prompt})

        # 체인 초기화
        chain = initialize_chain()

        # AI 응답 생성
        with st.chat_message("assistant"):
            with st.spinner("답변 생성 중..."):
                response = chain.invoke(
                    {"question": prompt},
                    {"configurable": {"session_id": "streamlit_session"}}
                )
                st.markdown(response)

        st.session_state.messages.append({"role": "assistant", "content": response})
```

```python
if __name__ == "__main__":
    main()
```

```python
# 터널링 및 실행
from pyngrok import ngrok

public_url = ngrok.connect(8501)  # Streamlit 기본 포트
print("앱 접속 URL:", public_url)
!streamlit run /content/app.py
```

터미널에서 streamlit run app.py 로 실행 명령어 대체 가능

pyngrok 라이브러리를 사용하여 스트림릿의 기본 포트인 8501에 대한 ngrok 터널을 생성. 이를 통해 외부에서 접근 가능한 공개 URL 생성.

```
앱 접속 URL: NgrokTunnel: "https://d628-34-23-186-167.ngrok-free.app" -> "http://localhost:8501"

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.28.0.12:8501
  External URL: http://34.23.186.167:8501
```

URL은 일반적으로 'https://[무작위주소].ngrok-free.app' 형식. 이 링크를 클릭하면 작성한 스트림릿 앱에 접근 가능.

URL에 접속하면 **ngrok 보안 페이지**가 먼저 표시됨. 보안 페이지에서 [Visit Site] 버튼을 클릭하면 스트림릿 앱이 로드됨.

KB 부동산 보고서 AI 어드바이저

2024 KB 부동산 보고서 기반 질의응답 시스템

안녕하세요.

안녕하세요! 어떻게 도와드릴까요?

수도권 주택 매매 전망

2024년 수도권 주택 매매 전망은 전반적으로 침체가 지속될 것으로 보입니다. 높은 기준금리와 주택 매매가격, DSR 규제 등으로 인해 매수자들의 구매 여력이 회복되지 않고 있으며, 매도자들은 정부의 규제 완화로 인해 기대 심리가 높아지고 있습니다.

특히, 강남구와 같은 선호 지역에서는 긍정적인 기대감이 유지되고 있지만, 전반적으로는 매도자와 매수자 간의 희망가격 격차가 여전히 존재하여 거래 회복이 제한적일 것으로 예상됩니다.

또한, 재건축 규제 완화와 광역급행철도(GTX) 개통 등 지역별 호재가 있지만, 전반적인 주택 경기 위축 요소도 여전히 존재하고 있습니다. 따라서, 수도권 주택시장은 지역별로 차별화된 흐름을 보일 것으로 예상되며, 매수세 회복이 중요한 요소로 작용할 것입니다.

부동산 관련 질문을 입력하세요